Ben Antonellis, Gabe Smith, Alex Nguyen

Privacy Preserving Smart Water

https://github.com/Linnydude3347/SmartWaterMeter

Preface

The way we monitor and manage vital resources has changed dramatically as a result of the incorporation of cutting edge technologies into critical infrastructure systems, such as water distribution networks. The idea of "smart water grids," similar to its smart energy grid counterpart, has gained traction recently as a viable way to improve the sustainability, dependability, and efficiency of water supply systems. Smart water grids use state-of-the-art technologies, such as data analytics and Internet of Things (IoT) devices, to maximize water distribution, find leaks, reduce cyber attacks, and enhance system performance.

A thorough examination of one such technology, fully homomorphic encryption, or FHE as it relates to smart water grids is provided in this document. With FHE, it is possible to protect sensitive data transported across smart water infrastructure and still use encrypted data for the operation of advanced analytics. The main goals, driving forces, and contributions of this research project are described in the introduction.

Introduction

Communities depend on water delivery systems as essential infrastructure because they guarantee the availability of clean water for many uses, including drinking, sanitation, and industrial use. Like its electrical counterparts, Advanced Metering Infrastructure (AMI) is essential to properly monitoring and controlling water consumption in contemporary smart water systems. The Internet of Things (IoT) is used in these smart water grids, which include data management systems, communication networks, and smart meters deployed at each customer's location.

Smart water networks are vulnerable to data integrity threats, which jeopardize the precision and dependability of usage data, just like electrical grids. Both utilities and customers are seriously at risk from these kinds of attacks. Similar to deductive assaults in electrical systems, data falsification in smart water grids frequently takes the form of water theft, when reported consumption figures are lower than the real usage. Additive attacks, on the other hand, entail manipulating reported consumption figures, which may result in increased expenses and improper use of resources. Additionally, more subtle threats come in the form of camouflage attacks, which combine additive and deductive attacks to maintain the mean consumption while secretly changing individual consumption records.

Ensuring the integrity, dependability, and effectiveness of smart water grids requires safeguarding user data while performing necessary computations for system analytics. Conventional systems often require access to individual consumption information, which raises privacy concerns and may breach privacy laws or damage customer confidence. Fully Homomorphic Encryption offers a solution by enabling secure computations on encrypted data, ensuring privacy is preserved without compromising analytical functionality.

Driven by the necessity to balance privacy concerns with computational requirements, researchers have investigated a range of privacy-preserving methods for analytics in smart water grids. While approaches like Secure Multiparty Computation and Differential Privacy provide theoretical guarantees of privacy, they often face scalability or accuracy challenges. Fully Homomorphic Encryption , in particular, offers a robust solution by enabling calculations to be performed directly on encrypted data without requiring decryption, ensuring both privacy and accuracy in data analytics.

Using a Look-Up Table (LUT) based FHE approach combined with Private Information Retrieval (PIR), we provide a privacy-preserving system inspired by the privacy-preserving system in smart energy grids designed for smart water grids in this research. By employing LUTs and PIR, our method addresses the key challenges of integrating FHE into smart water grids, providing an accurate solution that maintains user confidentiality. Early findings suggest that this innovative approach can enhance the integrity, dependability, and effectiveness of smart water grids. The following sections will detail the technology behind our method, explore implementation challenges, and present evidence of its impact on improving water management systems.

Glossary

Fully Homomorphic Encryption (FHE): A cryptographic technique that allows computations to be performed on encrypted data without the need for decryption, enabling privacy-preserving data processing and analysis.

Homomorphic Encryption (HE): A cryptographic scheme that supports certain algebraic operations on encrypted data, enabling computations to be performed on ciphertexts while still ensuring the correctness of the results.

Internet of Things (IoT): A network of interconnected devices embedded with sensors, software, and other technologies that enable them to collect and exchange data over the internet.

Private Information Retrieval (PIR): A cryptographic protocol that allows a user to retrieve information from a database without revealing which specific data items they are accessing.

Look-Up Table (LUT): A data structure used in computer science and cryptography to store precomputed values for efficient retrieval during computation.

Advanced Metering Infrastructure (AMI): A system of smart meters, communication networks, and data management systems that enable the remote monitoring and management of utility services, such as electricity or water consumption.

Differential Privacy (DP): A concept in data privacy that aims to provide strong guarantees for the protection of individual privacy in statistical databases or data analysis systems. The goal of DP is to enable the analysis of sensitive data while minimizing the risk of revealing information about specific individuals contained in the data.

Brakerski-Fan-Vercauteren (BFV): A fully homomorphic encryption scheme that enables computations to be performed on encrypted data. It is one of several FHE schemes that provide the ability to execute a range of arithmetic operations on ciphertexts such that, when decrypted, the result matches the outcome of the same operations performed on the plaintexts.

Single Instruction, Multiple Data (SIMD): Parallel computing paradigm where multiple data points are processed simultaneously under one instruction. This approach is useful when the same operation are used many times across a large dataset

User Requirements Definition

1. The system should be able to encrypt usage data in smart water grids while preserving the privacy of customer consumption data. The algorithms should operate directly on encrypted data without the need for decryption, ensuring that sensitive information remains protected.
2. Implementing Fully Homomorphic Encryption to encrypt customer consumption data at the point of collection would allow us to perform secure computations on the encrypted data to evaluate the accuracy of the encryption methods. The comparison between the encrypted and original data will be used to determine how closely the encrypted computations reflect the actual water usage data without needing to decrypt any additional private information.
3. Utilizing a homomorphic LUT-based approach to support privacy-preserving evaluation of data accuracy between encrypted and plaintext water usage data. We intend to store data pairs of input and output values in Look-Up Tables, facilitating the Arithmetic Mean-Harmonic Mean ratio calculations over a Fully Homomorphic Encryption scheme to measure how closely the encrypted computations match the original water consumption data, ensuring accuracy without decrypting private information.

4. Utilizing a LUT-based approach (since BFV doesn't support division) allows us to compute ratios. The BFV scheme's ability to perform additive and multiplicative operations on encrypted data makes it suitable for evaluating encrypted computations, facilitating accurate comparisons between encrypted and original water usage data without compromising privacy. Below, each communication component within the system architecture to be simulated:
    4.1. Smart Meters
    4.2. Data Collectors
    4.3. Computation Servers
    4.4. LUT Provider
    4.5. Utility Provider
5. The system shall implement functionality to demonstrate the accuracy of encrypted computations. This testing validates that Fully Homomorphic Encryption (FHE) preserves computational accuracy while maintaining data privacy.

System Requirements Specification

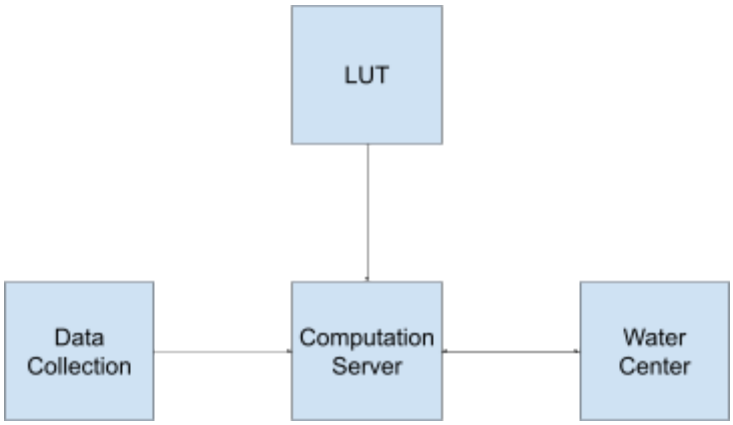
Microsoft SEAL: Cryptography library that performs operations on encrypted data. Supports FHE schemes including the Brakerski-Fan-Vercauteren scheme

CMake: Tool to assist in building processes of software

OpenMP: API that supports multi-platform shared memory multiprocessing which helps in multi-core processors work efficiently to enhance the performance of software

System Architecture

The experimental setup will be similar to that of the electrical grid experiment, having four main components. These components are the server where computations are performed, the LUT provider, the utility provider, and the region of water collection that has N smart meters. With this design each component will communicate with the computational server alone, and any necessary calculations will be gathered and done there. The figure below shows a basic representation of the setup simulation.



Smart Meter

- **Data Collection:** The smart meter will be responsible for collecting water consumption data from customers houses
- **Data Encryption:** Each smart meter will encrypt the collected data using a public key before transmission to ensure data remains confidential during transmission to the data collector

Data Collector

- **Data Hub:** The data collector will take in all the encrypted data from multiple smart meters across a region
- **Data Transmission:** The data will be forwarded to the computation server

Computation Server

- **Data Processing:** The computation server processes the encrypted data utilizing homomorphic encryption techniques

LUT Provider

- **LUT Maintenance:** Generates the Look-Up Tables that are essential to perform complex calculations on encrypted data
- **Complex Operations:** Provides support for operations not directly supported by homomorphic encryptions

Utility Provider/Water Center

- **Analysis:** Analyze the processed results sent by the computation server to detect any discrepancies

Operational Simulated Flow

- Key generations include public, secret, relinearization, and galios keys
    - Public Key: Initial encryption key
    - Secret Key: Decryption key
    - Relinearization Key: Manageable ciphertext sizes post multiplication operations
    - Galios Key: Rotations in SIMD processing and processes multiple data points simultaneously instead of needing to process one at a time
- Public keys will be shared with smart meters that encrypt the data at the source. Relinearization and Galios keys are sent to the computation server to support processing of encrypted data
- The public key encrypted water consumption information will be sent to the data collectors ensuring the information is protected
- The data collectors collect the encrypted data and forward all the information to the computation server
- The computation server performs all necessary computations including calculating the harmonic mean, arithmetic mean, and look up table
- The look up table is prepared by the LUT provider for operations that cannot be computed by the BFV scheme. The computation server utilizes the LUT to perform the necessary computations
- The processed data should remain encrypted throughout the entire computation process
- Final encrypted results is decrypted to compare the encrypted data to the original usage data

System Models

The HM and AM are fundamental metrics used to analyze water consumption within a given timeframe.

Arithmetic Formula:

$$AM_t = \frac{1}{N} \sum_{i=1}^{N} w_i(t)$$

Description: Calculates average water consumption over a defined period by summing individual meter readings and dividing by total numbers of meters

Harmonic Formula:

$$HM_t = \frac{N}{\sum_{i=1}^{N} \frac{1}{w_i(t)}}$$

Description: Average rates by taking the reciprocal of each rate

Ratio Over Time Slot Formula:

$$Q_d = \frac{\sum_{t \in T} HM_t}{\sum_{t \in T} AM_t}$$

Description: Dividing the harmonic mean from the arithmetic mean for a specific time slot

Daily Ratio Formula:

$$Q_d = \frac{\sum_{t=1}^{24} HM_t}{\sum_{t=1}^{24} AM_t}$$

Description: Dividing the harmonic mean from the arithmetic mean over all timeslots of the day

System Parameters

1. N: The number of water meters in the system.
2. T: Time Slots within a 24 hour period meaning every hour is a timeslot.
3. t: Specific time within set T.
4. wi(t): The water consumption reading from the i-th smart meter at timeslot t.
5. Qd: The HM over AM ratio for the d-th date/Timeslot.

BFV Characteristics

- Given the characteristics of the BFV scheme which only supports addition, subtraction, and multiplication, we must introduce a lookup table to perform the calculations for the ratio (division)

Prepared Look-Up Table for Ratio

Calculation	x (values in input table T_{in})	$f(x)$ (values in output table T_{out})
H_{Mt}	$\sum_{i=1}^{N} \left(\frac{1}{w_i(t)}\right)$	$f_1(x) = \frac{N}{x}$
A_{Mt}	$\sum_{i=1}^{N} w_i(t)$	$f_2(x) = \frac{x}{N}$
H_1	$\sum_{t=1}^{24} H_{Mt}$	$f_3(x) = \frac{x}{res2} \times 100$
H_2		$f_4(x) = x - f_3(x) \times 100$
A_1	$\sum_{t=1}^{24} A_{Mt}$	$f_5(x) = \frac{x}{100}$
A_2		$f_6(x) = x - f_5(x) \times 100$
$res2/100$		$f_7(x) = \frac{x}{100}$

Note: Using 100 as a placeholder value to scale up or down

Precision and Scaling

Adopting the LUT, we must calculate each time slot for HM and AM. Then we need to calculate the sums of HM and AM over a 24 hour period. HM/AM is how we compute Qd. We will be multiplying instead of dividing since the division function requires a two-input function.

Assume the maximum scaled ratio of Qd is 9 but the significant digits of the plaintext space is 5, we would have to discard the last 4 digits of the Qd result. Reducing the Qd result by 10000 to discard the last 4 digits since 100*100 = 10000.

HM/AM Ratio

$$Q_d = \frac{\sum_{t=1}^{24} HM_t}{\sum_{t=1}^{24} AM_t} = \sum_{t=1}^{24} HM_t \times \frac{1}{\sum_{t=1}^{24} AM_t}$$

Definition: The ratio of the sum of the harmonic means over the sum of the arithmetic means calculated over a 24 hour period. This ratio is computed using the multiplication of the inverse values due to limitations

Polynomial Representation

Multiplying the daily ratio of HM and AM makes the values too large which results in an overflow of the plaintext, so we drop the least significant digits off the result. Expressing HMt and AMt as a polynomial is shown below with H1, H2, A1, and A2 where these values are coefficients.

$$\sum_{t=1}^{24} HM_t = \mathcal{H}_1 \times 100 + \mathcal{H}_2$$

$$\frac{1}{\sum_{t=1}^{24} AM_t} = \mathcal{A}_1 \times 100 + \mathcal{A}_2$$

Note: 100 is not a fixed value, it depends on the plaintext space in the FHE setting and depends on how many significant digits we would like to keep. Larger plaintext space leads to more execution time. Dropping more significant bits allows us to reduce execution time

Daily Qd Ratio

$$Q^d = \sum_{t=1}^{24} HM_t \times \frac{1}{\sum_{t=1}^{24} AM_t}$$

$$
\begin{aligned}
&= (\mathcal{H}_1 \times 100 + \mathcal{H}_2) \times (\mathcal{A}_1 \times 100 + \mathcal{A}_2) \\
&= \mathcal{H}_1 \times \mathcal{A}_1 \times 10,000 \\
&\quad + (\mathcal{H}_1 \times \mathcal{A}_2 + \mathcal{H}_2 \times \mathcal{A}_1) \times 100 + \mathcal{H}_2 \times \mathcal{A}_2
\end{aligned}
$$

Definition: Due to size limitations in the plaintext space, using polynomial representation avoids overflow

Defining res1 and res2

$$res1 = \mathcal{H}_1 \times \mathcal{A}_1$$
$$res2 = \mathcal{H}_1 \times \mathcal{A}_2 + \mathcal{H}_2 \times \mathcal{A}_1$$

Discarding Least Significant Digits

$$Q^d = \text{res1} + \frac{\text{res2}}{100}$$

Note: This is scalability for the ratio formula

Definition: A necessary step in processing the Qd values where the last 4 digits are discarded to manage the size within the plaintext space

Precision Parameter and Vector Creation

Water consumption data will be retrieved by the smart meters, encrypted, and sent to the data collector. Using the LUT method, a packed ciphertext filled with elements that are identical to a search query, the smart meter will create two vectors v(wi(t)) and v((1/wi(t)) where wi(t) is water consumption:

$$v(w_i(t))[k] = \text{Round}(2^{p^{AMin}} \times w_i(t)),$$

$$v\left(\frac{1}{w_i(t)}\right)[k] = \text{Round}(2^{p^{HMin}} \times \frac{1}{w_i(t)}).$$

Note: 2p is a precision parameter, introducing this precision parameter allows us to understand trade-offs between execution time

Vector Length

Where 0 <= k < l, where l represents the length of the two vectors. Since we are employing integer encoding rather than bitwise encoding to speed up execution time, the data must be converted into integers before the encryption, allowing the data to be scaled with precision parameters 2p and rounded into integers.

Integer Encoding

- A method of encoding data into integers before encryption. This process enhances the efficiency of operations within cryptographic schemes that do not natively support high-precision floating-point arithmetic
- Converting data to integers reduces computational overhead which is essential when execution speed is a critical part

Encrypted Key Generation and Management

- Initiate Encryption Parameters
- The SEALContext object is created with the defined parameters
- The KeyGenerator object is created and responsible for generating public and private keys. Public keys are used for encryption and private keys are used for decryption
- Galios keys are generated to facilitate operations such as rotations in SIMD processing and utilize encrypted vectors to take advantage of multiple computations to reduce execution time
- Relinearization keys are generated for size reduction of ciphertexts after operations utilizing the
- Batching capabilities check to ensure batching is enabled to allow multiple values to be encoded into plaintext and then cipher text, allowing for parallel operations on multiple data values.
- Key serialization provides serialized encryption parameters that ensures the keys can be stored and distributed to the correct components of the system to perform encryptions, decryptions, and other homomorphic operations

Brakerski-Fan-Vercauteren Scheme

- Supports addition, subtraction, and multiplication
- Batching introduces the ability to have multiple values to be encoded into a single ciphertext which enhances computational efficiency
- Introducing relinearization keys help reduce degrees of polynomials down to a manageable size without the need of decrypting. Expanded ciphertext will be brought back down to either original size or degree after the initial multiplication. The relinearization keys are essential to performing sequences of multiplication without the ciphertexts becoming too large.
- The BFV scheme can easily be adjusted to balance between anomaly detection and performance making it scaleable to small and large data sets
- Noise Management can be an issue when performing homomorphic operations on ciphertext, BFV has efficient noise growth management which is especially critical for accurate computations with homomorphic operations

System Evolution

Initial Concept

This project initially started with the concept of enhancing privacy in smart water grids through encryption. The idea was to integrate cryptographic methods to protect customer water consumption data while allowing necessary computational analytics. There were currently many different cryptographic schemes that could've been used. We ultimately went with Fully Homomorphic Encryption at the discretion of our client

First Implementation

We were provided a codebase thats main purpose was to encrypt smart energy usage data, perform the necessary analytical computations, and decrypt the data. We inherited this codebase while focusing on the same cryptographic techniques to ensure the program was functional

Dataset Shift

Given that the codebase we inherited was focused on smart energy usage, we had to shift the codebase to focus on smart water usage. This required some adaptations in the dataset handling to ensure that the code was functional and producing the right output

Testing Accuracy and Validation

To ensure that the encrypted data computations were consistent with plaintext, we had to conduct accuracy tests. This phase involved comparison between computations on ciphertexts and plaintext data

Summary of Evolution

The evolution of this privacy-preserving smart water project involved careful integration of the Fully Homomorphic Encryption technology with data analytics. Starting from the proof of concept, to being provided the initial codebase of the electrical counterpart, the system evolved to overcome challenges of encrypted computations.

Conclusion

This project successfully demonstrates a privacy-preserving approach to analyzing smart water consumption data using Fully Homomorphic Encryption (FHE). Through the implementation of the BFV scheme, we were able to encrypt water usage data while still performing complex analytics, such as calculating the Harmonic Mean (HM), Arithmetic Mean (AM), and their ratio, directly on encrypted data. This approach ensured that customer privacy was maintained throughout the entire data processing workflow, providing a solution that is both secure and effective for smart water systems.

Key Achievements

- **Privacy Preserving**
- **Accuracy and Security**
- **Efficient Computations**

Future Direction

This project could focus more on scalability improvements to explore the potential for more efficient LUT and PIR integrations into larger data sets. This projects concept can be extended into measuring advanced analytics and measuring more advanced metrics as well. This project also could expand into other industries as well demonstrating the versatility of a Fully Homomorphic Encryption infrastructure

Appendices

Appendix A: Key Generation

Public Key

Purpose: The public key is used to encrypt data at the point of collection. For this project, each smart water meter uses the public key to encrypt the water usage data. This ensures that no one can see the unencrypted data

Usage: The public key is shared with smart water meters across a network enabling them to encrypt data at the source of collection

Secret Key

Purpose: The secret key is used to decrypt the data after computations are performed on ciphertexts. This key allows to recover the processed plaintext data
Usage: The secret key is retained securely by the party authorized to decrypt the data and in the projects focus, the utility provider

Relinearization Key

Purpose: This key is used to manage and reduce sizes of ciphertexts after performing multiplicative operations. When two encrypted values are multiplied the ciphertext becomes more complex resulting in higher polynomial degrees. This key reduces the key back to a manageable size allowing computations to be performed more efficiently
Usage: This key is used by the Computation Server to ensure efficient processing of ciphertexts after multiplicative operations to keep ciphertexts from growing uncontrollably

Galois: Key

Purpose: This key is used to enable Single Instruction, Multiple Data operations such as vector rotations on encrypted data. This allows parallel processing of multiple data at once improving computational efficiency
Usage: Galois keys are used when rotating encrypted vectors. In this project is allows operations such as shifting or rotating data within ciphertext
The keys will be stored in the Keys directory, and must be loadable from different contexts as long as they use the same encryption parameters. This aims to reduce the complexity of re-generating keys for every step.

Appendix B: System Setup & Components

Requirements (Exact versions are needed)

Microsoft SEAL 4.1.2

OpenMP 18.1.8

CMake 3.10

Optional Dependencies

For testing Microsoft SEAL yourself, you may use these optional dependencies:
GoogleTest 1.12.1
GoogleBenchmark 1.7.1

Getting Started

Download this zip file and ensure you have the correct versions of the required software before running. When building manually, you might run into an issue of CMake not identifying the software installed on your computer. In that case, navigate to src/1hour and open CMakeLists.txt.

Under the comment # Import OpenMP, you can set the specific install path of the required software. Ensure you correct every path.

Appendix C: Technical Implementation

This appendix provides an indepth overview of the technical aspect and implementation details for every file needed within the project. Every file covers all the necessary cryptographic techniques along with the practical steps required to implement this project correctly

CheckRes.cpp

Purpose: Validates accuracy of encrypted computations

KeyGen.cpp

Purpose: Generates and saves cryptographic keys for encryption

Step1_CS1.cpp

Purpose: Calculates the Arithmetic Mean and Harmonic Mean from the dataset

Step2_TA1.cpp

Purpose: Decrypts and stores the AM and HM data for analysis

Step3_CS2_1.cpp

Purpose: Processes encrypted Arithmetic Mean results using PIR

Step3_CS2_2.cpp

Purpose: Processes encrypted Harmonic Mean results using PIR

Step4_TA2.cpp

Purpose: Retrieves, decrypts, and decodes saved Arithmetic and Harmonic mean results

Step5_CS3_1.cpp

Purpose: Handles Arithmetic Mean data computations and saves results

Step5_CS3_2.cpp

Purpose: Handles Harmonic Mean data computations and saves results

Step6_TA3.cpp
Purpose: Verifies encrypted data consistency between Arithmetic mean and Harmonic mean

Step7_TA4.cpp
Purpose: Performs homomorphic operations on encrypted data using LUT and PIR, and computes final results and saves for analysis

Index
A
Arithmetic Mean - 6

B
Brakerski-Fan-Vercauteren - 3, 11

F
Fully Homomorphic Encryption - 2

H
Harmonic Mean - 6

Bibliography

[1] Smart Grid Detection Simulation
[2] IEEE Privacy Preserving Analytics Publication
[3] Professor Shameek PPA Paper
[4] Professor Shameek Forensics Paper
[5] Detection of False Data Injection in Smart Water Metering Infrastructure